TECNOLÓGICA DEL ORIENTE
INGENIERÍA DE SOFTWARE

PRUEBA Y CALIDAD DE SOFTWARE — GRUPO N6 A
UNIDAD 2 — ACTIVIDAD AA2

TALLER GRUPAL: AUTOMATIZACIÓN DE PRUEBAS CON JUNIT, SELENIUM O POSTMAN

Pruebas unitarias, de API y de interfaz web, cobertura de código e integración continua

Presentado por: Carlos Arturo Rueda Calier
Modalidad: individual
Presentado a: Docente de Prueba y Calidad de Software
Repositorio: github.com/CarlosDev88/calidad-software-u2-scripts-automatizados

Piedecuesta, Santander, 11 de noviembre de 2026

CONTENIDO

1. INTRODUCCIÓN Y OBJETIVO 3

2. PUNTO DE PARTIDA Y ALCANCE 3

3. APLICACIÓN BAJO PRUEBA 3

4. HERRAMIENTAS, ALTERNATIVAS Y JUSTIFICACIÓN 4

5. GUÍA PASO A PASO 4

6. PRUEBAS UNITARIAS CON JUNIT 5 Y MOCKITO 5

7. PRUEBAS DE API REST 6

8. PRUEBAS DEL FORMULARIO WEB CON SELENIUM WEBDRIVER 7

9. DEFECTOS ENCONTRADOS Y CORREGIDOS 9

10. DIFICULTADES DEL PROCESO 9

11. COBERTURA Y EFICIENCIA DE LA SUITE 10

12. INTEGRACIÓN CONTINUA CON GITHUB ACTIONS 11

13. RESULTADOS CONSOLIDADOS 12

14. ORGANIZACIÓN DEL TRABAJO 13

15. REFLEXIÓN FINAL 13

REFERENCIAS 14

1. INTRODUCCIÓN Y OBJETIVO

Este informe documenta la automatización de pruebas funcionales de una aplicación web pequeña de registro de usuarios, formada por una API REST y un formulario web. El trabajo parte del proyecto de la actividad AA1, en el que se diseñaron scripts para la API y el formulario, y lo amplía con pruebas unitarias con JUnit 5 y Mockito, más casos de API con TestRestTemplate y Postman, pruebas de interfaz con Selenium WebDriver, medición de cobertura con JaCoCo y ejecución automática en GitHub Actions.

El objetivo es mostrar, con código, comandos y resultados reales, cómo se construyó y se ejecuta la suite en los tres niveles de prueba, qué decisiones se tomaron y qué problemas aparecieron en el camino, incluidos tres defectos de la aplicación que se encontraron y corrigieron durante el taller. Las cifras provienen de la carpeta *evidencias/* del repositorio, generada con Docker Desktop en el equipo del estudiante el 11 de noviembre de 2026, y de las ejecuciones de GitHub Actions.

2. PUNTO DE PARTIDA Y ALCANCE

El repositorio se trabajó como una continuación del AA1 y no como un proyecto nuevo, para reutilizar la aplicación, los datos de prueba y la infraestructura ya validados.

Aspecto	AA1 (punto de partida)	AA2 (este taller)
Niveles de prueba	API e interfaz	Unitaria, API e interfaz
Herramientas	JUnit 5, TestRestTemplate, Selenium, Postman	Se agregan Mockito, MockMvc y Bean Validation
Pruebas JUnit	21	62
Técnicas de diseño	Casos válidos e inválidos	Valores límite y particiones de equivalencia
Defectos	Ninguno reportado	Tres encontrados y corregidos (sección 9)
Cobertura	Sin medición	JaCoCo con mínimo obligatorio del 80 %
Ejecución	Manual con Docker	Docker y GitHub Actions en cada push
Documentación	README	README y GUIA_PASO_A_PASO.md

3. APLICACIÓN BAJO PRUEBA

La aplicación está hecha con Spring Boot y guarda los usuarios en memoria, lo que permite repetir las pruebas sin preparar una base de datos. Expone tres operaciones y un formulario *registro.html* que envía los datos a la API y muestra los errores debajo de cada campo.

Operación	Ruta	Respuestas
GET	/api/usuarios	200: lista de usuarios
GET	/api/usuarios/{id}	200: usuario; 404: no existe; 400: id no numérico
POST	/api/usuarios	201: creado; 400: datos inválidos, tipo de dato inválido o JSON mal formado; 409: email duplicado
Otros	Método o Content-Type no soportado	405 y 415

Reglas de negocio: el nombre es obligatorio y tiene de 3 a 60 caracteres sin contar los espacios del inicio y del final; el email es obligatorio, con formato válido y sin repetirse; la edad es obligatoria, entera y va de 18 a 100. Todas las respuestas de error anteriores, incluidas 405 y 415, usan un mismo formato JSON (estado, mensaje y error por campo), definido en la clase *ManejadorErrores*.

4. HERRAMIENTAS, ALTERNATIVAS Y JUSTIFICACIÓN

Se siguió la pirámide de pruebas (Fowler, 2012): muchas pruebas unitarias rápidas, menos de integración y pocas de interfaz, que son las más lentas. Para cada necesidad se comparó al menos una alternativa:

Necesidad	Elegida	Alternativa	Razón de la elección
Unitarias	JUnit 5 + Mockito (Spring Boot 3.3.5)	TestNG	Viene con Spring Boot; @ParameterizedTest y @Nested se usaron mucho (JUnit Team, s. f.; Mockito, s. f.).
API desde Java	TestRestTemplate	REST Assured	La rúbrica la nombra y viene incluida (Spring, 2024). REST Assured es más legible pero suma una dependencia.
API externa	Postman + Newman 6.1.3	Insomnia	Lo pide la actividad; Newman corre la misma colección en consola y en CI (Postman, s. f.).
Interfaz	Selenium WebDriver 4.25.0	Playwright	Lo pide la actividad. Playwright espera los elementos solo; con Selenium hubo que programar esperas explícitas.
Cobertura	JaCoCo 0.8.12, mínimo 80 %	Sin mínimo	Un umbral común: obliga a probar casi todo sin exigir 100 % en cada clase (Mountainminds GmbH & Co. KG, s. f.).
CI	GitHub Actions	Jenkins	El código ya está en GitHub y no hay que montar un servidor (GitHub, s. f.).
Entorno	Docker Compose	Instalación local	Versiones fijas: mismo resultado en cualquier equipo (Docker Inc., s. f.).

ESTRUCTURA DEL ENTORNO

```
Equipo con Docker Desktop (Windows)
 +-- chrome : selenium/standalone-chrome:4.25.0-20240922
 |            navegador + Selenium Grid (puerto 4444; vista en vivo en el 7900)
 +-- tests  : maven:3.9.9-eclipse-temurin-21
 |            mvn verify: levanta la app, corre las 62 pruebas JUnit y la cobertura
 +-- app    : misma imagen; la app en el puerto 8080 (perfiles "demo" y "postman")
 +-- newman : postman/newman:6.1.3-alpine; corre la coleccion contra http://app:8080
```

El contenedor de pruebas levanta la aplicación en un puerto aleatorio y Selenium la abre desde el contenedor de Chrome usando el nombre del servicio (*tests*) como dirección. Se usó una aplicación propia en lugar de un sitio público: una página externa puede cambiar o caerse y la prueba fallaría sin que exista un error real.

5. GUÍA PASO A PASO

La guía completa está en *GUIA_PASO_A_PASO.md* y tiene las mismas dos partes que esta sección: cómo se construyó la suite y cómo se ejecuta.

PARTE A. CÓMO SE CONSTRUYÓ

A1. Reglas y casos. Por cada regla se definieron valores válidos, inválidos y límites, con dos técnicas de caja negra (Myers et al., 2011; International Software Testing Qualifications Board [ISTQB], 2023): **valores límite**, probando justo en el borde y justo afuera, y **particiones de equivalencia**, con un representante por cada grupo de entradas que se comporta igual.

Regla	Valores límite	Particiones de equivalencia
Edad 18 a 100	17 (inválido), 18, 100 (válidos), 101 (inválido)	Nula; negativa; decimal (18.9); texto ("25"); dentro del rango
Nombre 3 a 60	2 (inválido), 3, 60 (válidos), 61 (inválido)	Vacío; solo espacios; " A "; normal
Email	—	Válido simple; con punto y +; sin @; sin dominio; sin usuario; dominio sin punto

A2. Organización por nivel. Cada clase lleva una etiqueta *@Tag("unit")*, *@Tag("api")* o *@Tag("ui")* para poder correr solo una parte de la suite.

```
src/test/java/co/edu/calidad/taller/
  unit/    UsuarioServiceTest, UsuarioRequestValidacionTest, UsuarioControllerTest   (34 pruebas)
  api/     UsuarioApiTest                                                            (20 pruebas)
  web/     RegistroFormularioTest + pages/RegistroPage + support/DriverFactory        (8 pruebas)
  support/ DatosPrueba, Evidencias, ReporteResultados                               (utilidades)
```

A3 a A6. Se escribieron las pruebas unitarias, de API y de formulario (secciones 6 a 8), se configuró JaCoCo en *pom.xml* y el flujo *.github/workflows/pruebas.yml*. **A7.** Las dificultades y defectos encontrados están en las secciones 9 y 10.

PARTE B. CÓMO SE EJECUTA (POWERSHELL EN WINDOWS)

Comandos usados para generar las evidencias de este informe:

```
git clone https://github.com/CarlosDev88/calidad-software-u2-scripts-automatizados.git
cd calidad-software-u2-scripts-automatizados

# B2. Todas las pruebas JUnit (unitarias + API + Selenium) y la cobertura
docker compose up --build --exit-code-from tests

# B3. Solo un nivel (en PowerShell las comillas son necesarias)
docker compose run --rm tests mvn -B test "-Dgroups=unit" "-Djacoco.skip=true"

# B4. Coleccion Postman con Newman, guardando la salida
docker compose --profile postman run --rm newman | Tee-Object evidencias\salida-newman.txt
docker compose --profile demo --profile postman down

# B5. Reporte de cobertura
start evidencias\cobertura\html\index.html
```

B6. Al hacer push a *main*, GitHub Actions corre el mismo comando de B2 (sección 12). **B7.** La carpeta *evidencias/* queda con: *resultados-pruebas.txt* (cada prueba con OK o FALLO), *salida-consola.txt*, *surefire-reports/*, *cobertura/*, *capturas/* (14 imágenes de Selenium), *newman-resultados.xml*, *salida-newman.txt* y *defectos-antes-de-corregir.txt*.

6. PRUEBAS UNITARIAS CON JUNIT 5 Y MOCKITO

Son 34 y se dividen en tres clases:

- **UsuarioServiceTest** (6): crea el servicio con *new*, sin levantar Spring, y agrupa con *@Nested* ("Al crear", "Al consultar"): ids consecutivos, email duplicado sin importar mayúsculas, id inexistente y limpieza.
- **UsuarioRequestValidacionTest** (22): aplica los valores límite y las particiones del paso A1 directamente sobre las reglas, con *@ParameterizedTest*, *@CsvSource* y *@ValueSource*.
- **UsuarioControllerTest** (6): *@WebMvcTest* levanta solo la capa web y el servicio se reemplaza por un objeto simulado de Mockito, para verificar que el controlador traduce cada situación al código HTTP correcto y que con datos inválidos no llama al servicio.

Método completo de UsuarioRequestValidacionTest.java:

```
@ParameterizedTest(name = "edad {0} -> valida: {1}")
@CsvSource({
        "-1,  false",   // particion: edad negativa
        "17,  false",   // justo debajo del minimo
        "18,  true",    // minimo
        "100, true",    // maximo
        "101, false"    // justo encima del maximo
})
@DisplayName("Limites de la edad (18 a 100)")
void limitesEdad(int edad, boolean esValida) {
    Set<String> errores = camposInvalidos(new UsuarioRequest("Ana Perez", "ana@correo.com", edad));

    assertThat(errores.contains("edad")).isEqualTo(!esValida);
}
```

Método completo de UsuarioControllerTest.java (solo se partió una línea larga):

```
@Test
@DisplayName("POST invalido responde 400 y NUNCA llama al servicio")
void crearInvalidoNoLlamaAlServicio() throws Exception {
    String jsonInvalido = """
            {"nombre": "", "email": "ana-correo", "edad": 15}
            """;

    mockMvc.perform(post("/api/usuarios")
                    .contentType(MediaType.APPLICATION_JSON).content(jsonInvalido))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errores.nombre").value("El nombre es obligatorio"))
            .andExpect(jsonPath("$.errores.email").exists())
            .andExpect(jsonPath("$.errores.edad").value("La edad minima es 18"));

    verify(servicio, never()).crear(any());      // Mockito: el servicio no se uso
}
```

7. PRUEBAS DE API REST

TESTRESTTEMPLATE

UsuarioApiTest usa *@SpringBootTest(webEnvironment = RANDOM_PORT)*: levanta la aplicación real y hace peticiones HTTP verdaderas (Spring, 2024). Antes de cada caso limpia los datos. Son 20 pruebas:

Caso y entrada	Aserción principal	Resultado
POST válido (Ana, email único, 25)	201; id generado; datos iguales; cabecera Location	OK
GET lista con dos usuarios / GET id existente / GET id 999	200 lista de 2 / 200 mismo usuario / 404 con el id	OK x3
POST inválido x10: nombre vacío, "Al", " A ", solo espacios; email "ana-correo" y vacío; edad 15, 120, nula y -1	400; error en el campo correcto; no se guarda nada	10 OK
POST con edad 18.9 y con edad "25" (texto)	400; no se guarda nada	2 OK
DELETE /api/usuarios/1 y POST con text/plain	405 y 415 con el formato de error de la API	2 OK
POST email repetido / JSON roto	409 y queda un usuario / 400 "mal formada"	2 OK

La colección tiene siete peticiones en orden: crear (201), consultar por id (200), listar (200), email inválido (400), menor de edad (400), email duplicado (409) e id inexistente (404). La primera guarda el id y el email en variables que usan las siguientes, y a nivel de colección hay una prueba común: respuesta en menos de 2 segundos y en formato JSON (Postman, s. f.). En el equipo del estudiante: 7 peticiones y 29 aserciones, sin fallos, con un tiempo promedio de respuesta de 30 ms.

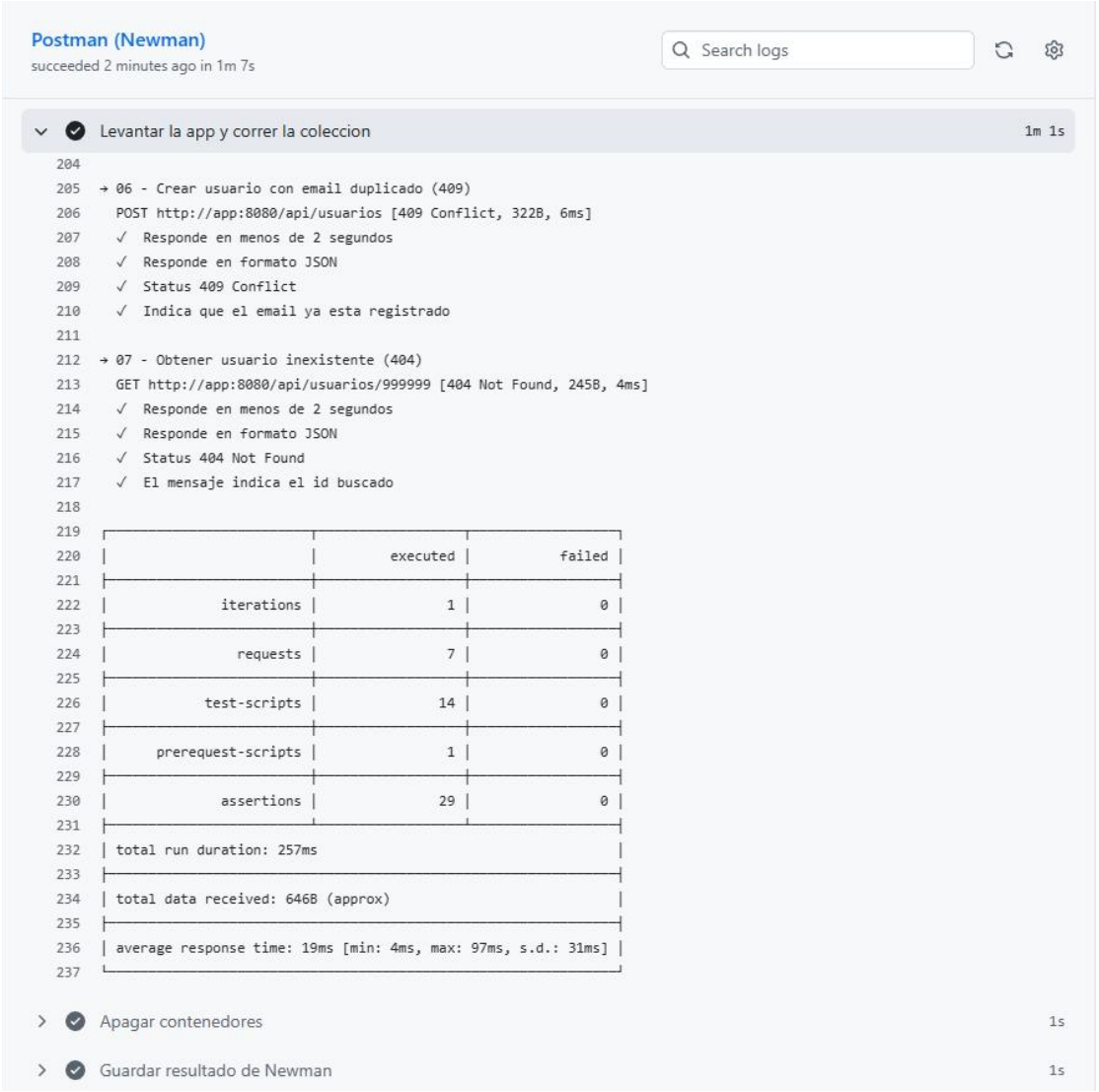


Figura 1. Ejecución de la colección con Newman en GitHub Actions (trabajo "Postman (Newman)"): 7 peticiones y 29 aserciones sin fallos.

8. PRUEBAS DEL FORMULARIO WEB CON SELENIUM WEBDRIVER

RegistroFormularioTest abre Chrome, llena el formulario y lee los mensajes que ve el usuario. Se aplicaron prácticas recomendadas por la documentación de Selenium (Selenium Project, s. f.-a, s. f.-b; Fowler, 2013):

- **Page Object**: *RegistroPage* concentra localizadores y acciones; si cambia el HTML se corrige en un solo lugar.
- **Esperas explícitas**: *WebDriverWait* de hasta 10 segundos espera el mensaje, en lugar de pausas fijas.

- **Evidencia automática**: cada caso guarda una captura de los datos de entrada y otra del resultado; si falla, *CapturaEnFallo* guarda la pantalla antes de cerrar el navegador.

Caso y entrada	Aserción principal	Resultado
Registro válido (Ana, email único, 25)	Mensaje de éxito; nombre, email y edad vacíos	OK
Email "ana-correo"	Error de formato bajo el campo email	OK
Nombre vacío / nombre "Al"	Error de obligatorio / de longitud	OK / OK
Edad 15 / edad 120	Error de edad mínima / máxima	OK / OK
Formulario vacío	Error en los tres campos	OK
Email repetido tras un registro	Error de email ya registrado	OK

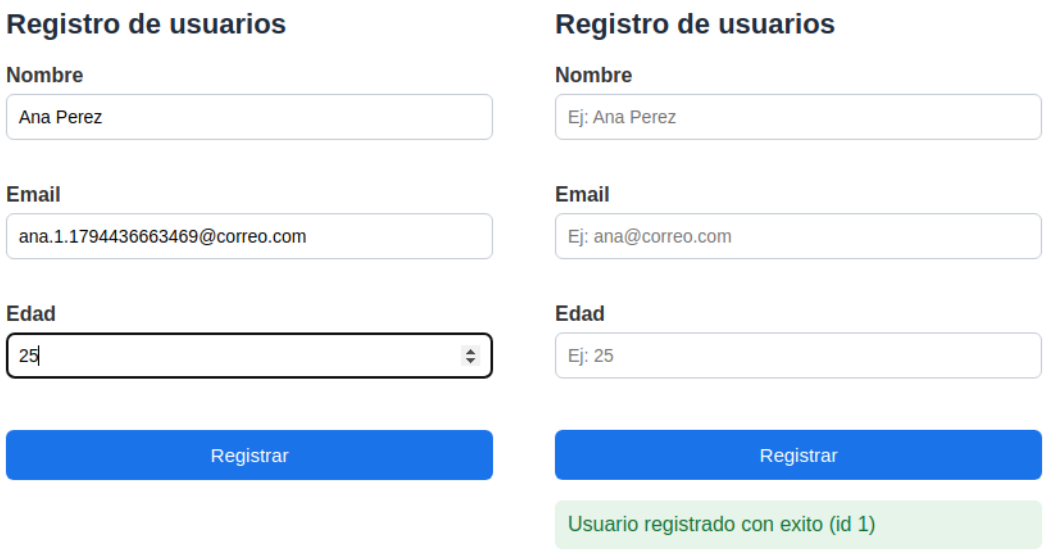


Figura 2. Capturas tomadas por Selenium en Docker. Registro válido: datos de entrada y resultado con el formulario limpio.

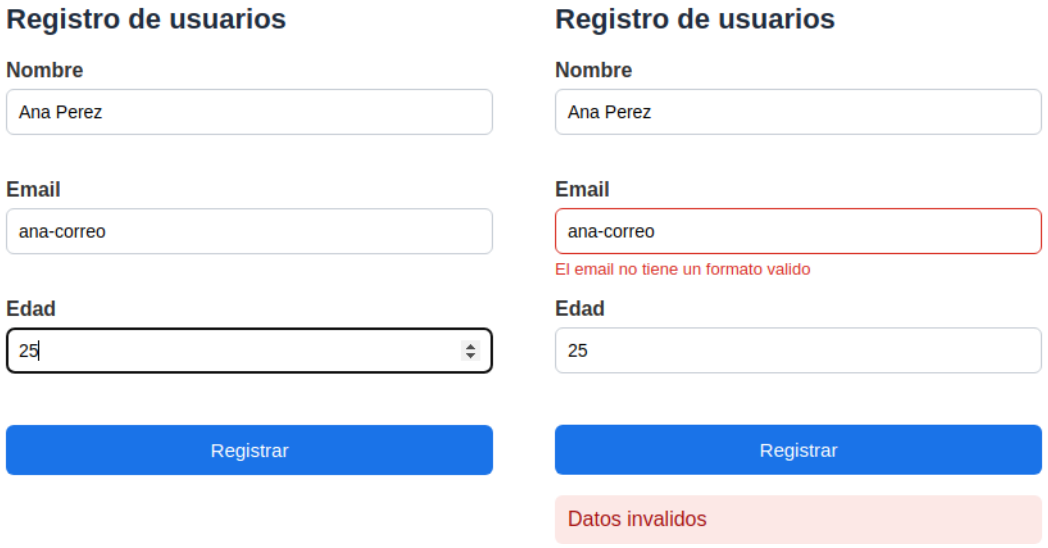


Figura 3. Email con formato inválido: datos de entrada y error mostrado bajo el campo.

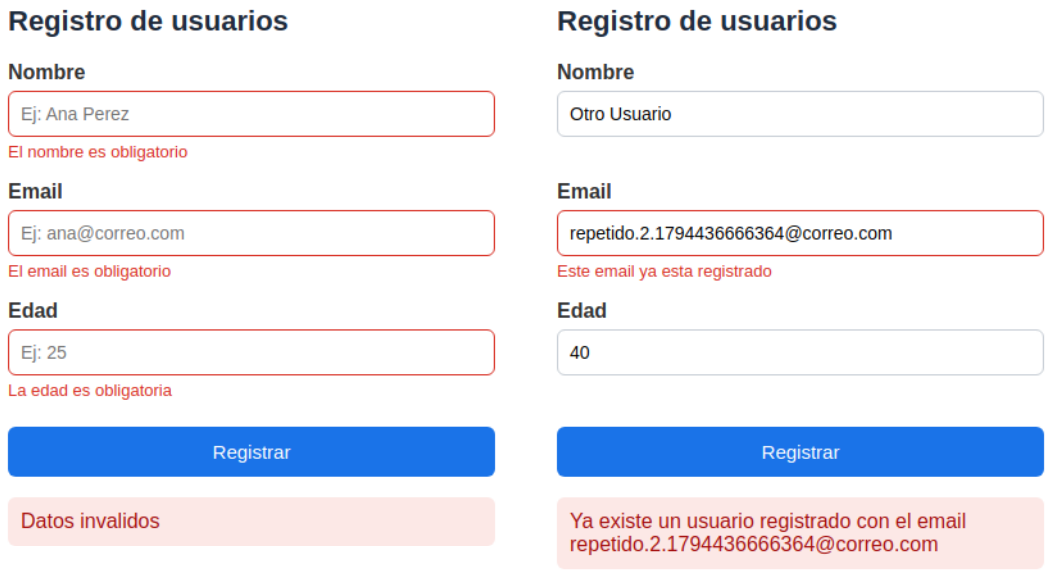


Figura 4. Formulario vacío con error en los tres campos (izquierda) y email duplicado (derecha).

9. DEFECTOS ENCONTRADOS Y CORREGIDOS

Después de tener la suite en verde, probé la API a mano con casos que no estaban en las pruebas y aparecieron tres defectos reales. Antes de tocar el código escribí las pruebas que los reproducen; 8 de 55 fallaron, lo que confirmó los defectos. Luego corregí el código y la suite volvió a verde.

Defecto	Causa	Corrección
" A " pasaba la regla de 3 a 60 caracteres y se guardaba como "A"	El servicio quitaba los espacios después de validar	Los espacios se quitan al crear *UsuarioRequest*, antes de validar
Edad 18.9 se guardaba como 18 y "25" (texto) se aceptaba	Jackson convierte tipos en silencio	*accept-float-as-int=false* y *allow-coercion-of-scalars=false*
405 y 415 respondían con otro formato JSON	Solo se manejaban 400, 404 y 409	Dos manejadores nuevos en *ManejadorErrores*

Resumen de evidencias/defectos-antes-de-corregir.txt (se unieron en una línea los mensajes de cada falla):

```
$ mvn test -Dgroups=unit,api   (pruebas nuevas escritas ANTES de corregir el codigo)
[ERROR]   UsuarioApiTest.metodoNoSoportado:187         expected: 405  but was: 0
[ERROR]   UsuarioApiTest.rechazarDatosInvalidos:144    expected: 400 BAD_REQUEST  but was: 201 CREATED
[ERROR]   UsuarioApiTest.rechazarEdadNoEntera:175      expected: 400 BAD_REQUEST  but was: 201 CREATED  (x2)
[ERROR]   UsuarioApiTest.tipoDeContenidoNoSoportado:202 expected: 415  but was: 0
[ERROR]   UsuarioRequestValidacionTest.nombreConEspaciosSeValidaRecortado:115  (x2)
[ERROR]   UsuarioRequestValidacionTest.recortaEspaciosAntesDeValidar:123
            expected: "Ana Perez"  but was: "  Ana Perez  "
[ERROR] Tests run: 55, Failures: 8, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

El caso "but was: 0" en 405 y 415 significa que la respuesta llegaba sin el campo *estado* del formato de la API: el código HTTP era correcto, pero el cuerpo tenía el formato por defecto de Spring.

10. DIFICULTADES DEL PROCESO

Problema	Causa	Solución
Un mismo dato inválido devolvía a veces un mensaje y a veces otro	"" falla @NotBlank y @Size a la vez y el orden no está garantizado	El mensaje de "obligatorio" tiene prioridad en ManejadorErrores
La imagen postman/newman:6.2.1-alpine no existía	Se asumió una etiqueta sin verificarla	Se usó 6.1.3-alpine, la 6.x publicada más reciente
Las primeras conexiones a Selenium fallaban a veces	El contenedor de Chrome tarda en estar listo	DriverFactory reintenta la conexión hasta 10 veces
Fuera de Docker Chrome no abría	ChromeDriver y Chrome de versiones distintas	Variables CHROME_BIN y CHROMEDRIVER_PATH
En PowerShell, -Dgroups=unit daba error de Maven	PowerShell parte el argumento en el signo =	Escribirlo entre comillas: "-Dgroups=unit"
En GitHub Actions las pruebas pasaban pero fallaba subir las evidencias (EACCES)	Las capturas quedaban con permisos solo para root dentro del contenedor	Evidencias escribe los bytes directamente y el flujo da permisos de lectura

11. COBERTURA Y EFICIENCIA DE LA SUITE

JaCoCo mide la cobertura durante las pruebas y *mvn verify* falla si la cobertura de líneas baja del 80 % (Mountainminds GmbH & Co. KG, s. f.). Solo se excluyó la clase que arranca Spring. Resultado: 100 % de líneas (51 de 51), de instrucciones (282 de 282) y de ramas (18 de 18), en 8 clases.

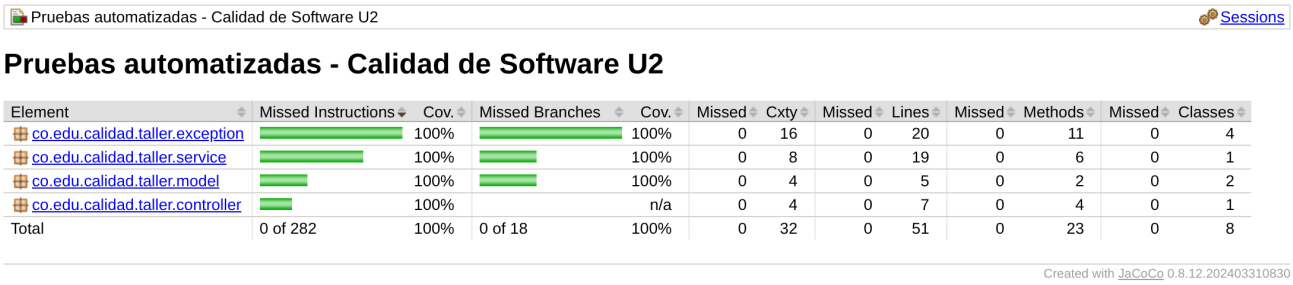


Figura 5. Reporte de JaCoCo generado en Docker en el equipo del estudiante (evidencias/cobertura/html/index.html).

El 100 % indica que cada línea se ejecutó al menos una vez, no que no haya errores: el defecto del nombre con espacios estaba en código que ya tenía cobertura completa. Por eso importan más las aserciones y los casos límite que el porcentaje.

Tiempos tomados de los reportes JUnit de la ejecución en Docker:

Nivel	Pruebas	Tiempo (s)	Por prueba
Unitarias (JUnit 5 + Mockito)	34	2,12	0,06 s
API (TestRestTemplate)	20	0,76	0,04 s
Interfaz (Selenium)	8	24,84	3,11 s
Total JUnit	62	27,72	—
Postman (Newman)	7 peticiones / 29 aserciones	0,44	30 ms promedio por respuesta

Los datos confirman la pirámide: las unitarias son más de la mitad de la suite y tardan dos segundos, mientras que ocho pruebas de navegador concentran casi el 90 % del tiempo.

12. INTEGRACIÓN CONTINUA CON GITHUB ACTIONS

.github/workflows/pruebas.yml tiene dos trabajos que corren en cada push a *main* (GitHub, s. f.). El primero ejecuta el mismo comando Docker de la guía y guarda las evidencias como archivo descargable; el segundo corre la colección Postman y solo se ejecuta si el primero pasa.

Fragmento de pruebas.yml (se omiten los nombres de los pasos y las opciones de upload-artifact):

```
jobs:
  junit-selenium:                                  # JUnit + Selenium + cobertura (Docker)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose up --build --exit-code-from tests
      - run: docker compose down
      - run: sudo chmod -R a+rX evidencias         # ver seccion 10
      - uses: actions/upload-artifact@v4           # evidencias descargables
  postman:
    needs: junit-selenium
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose --profile postman run --rm newman
```

La primera ejecución con este flujo (#2) falló en el paso de subir evidencias por el problema de permisos de la sección 10; después de corregirlo, la ejecución #3 terminó con éxito en 2 min 49 s.

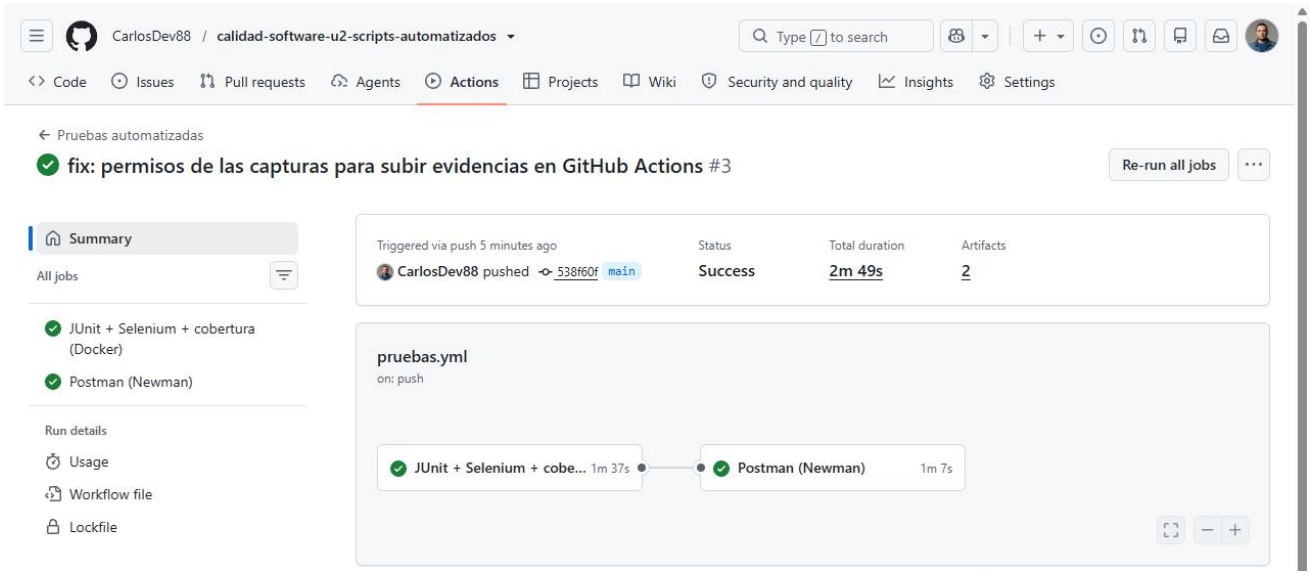


Figura 6. Resumen de la ejecución #3 en GitHub Actions: los dos trabajos terminaron con éxito y se generaron dos archivos de evidencias.

13. RESULTADOS CONSOLIDADOS

Fuente de evidencia	Ejecutadas	Aprobadas	Fallos / errores
JUnit unitarias	34	34	0 / 0
JUnit API (TestRestTemplate)	20	20	0 / 0
JUnit formulario (Selenium)	8	8	0 / 0
Total JUnit	62	62	0 / 0
Newman: peticiones / aserciones	7 / 29	7 / 29	0 / 0
Cobertura de líneas (mínimo 80 %)	51 líneas	51 (100 %)	Regla cumplida

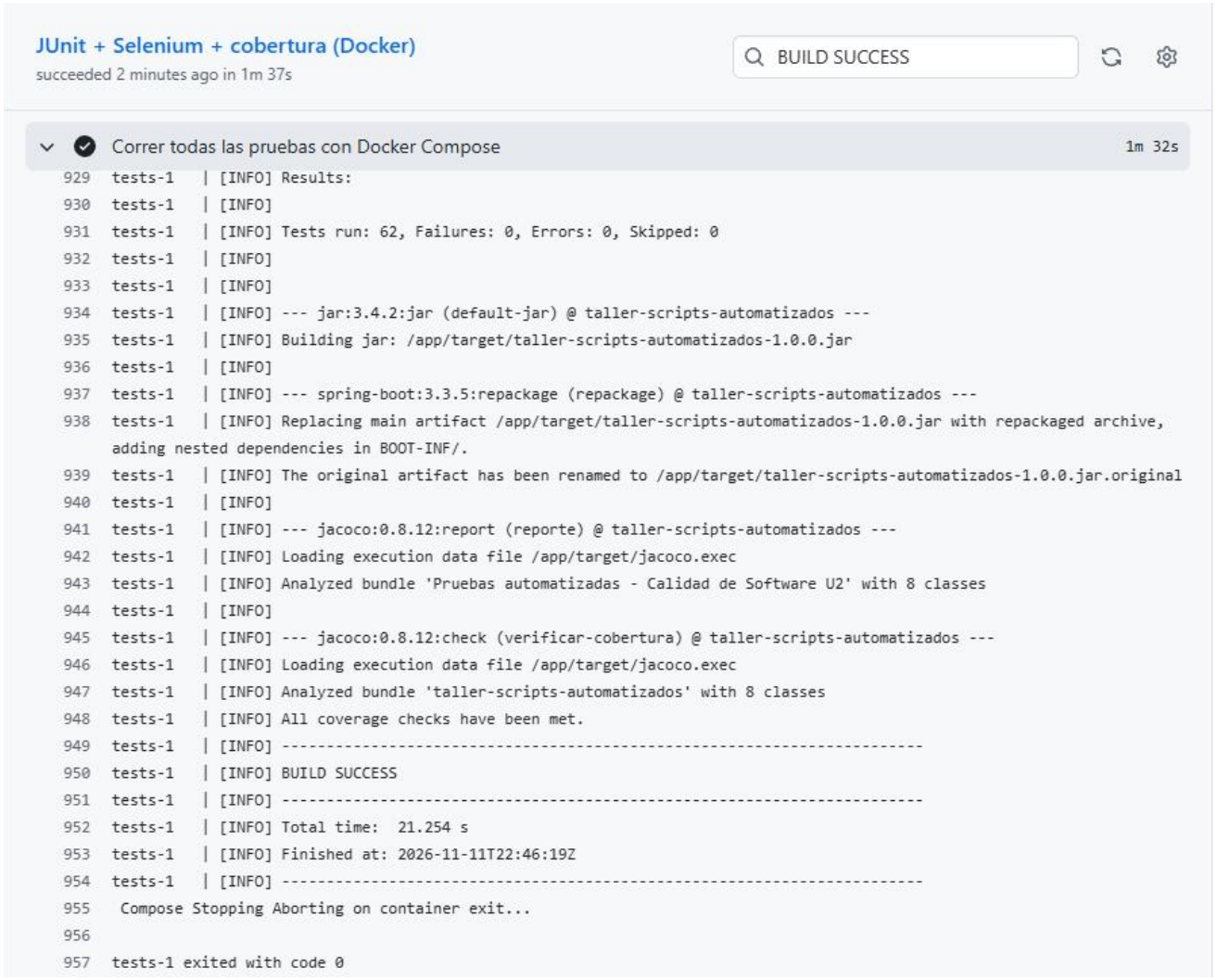


Figura 7. Final del paso "Correr todas las pruebas con Docker Compose" en GitHub Actions: 62 pruebas sin fallos, cobertura cumplida y BUILD SUCCESS.

Extracto de evidencias/salida-consola.txt (ejecución con Docker en el equipo del estudiante; hora en UTC):

```
[INFO] Tests run: 62, Failures: 0, Errors: 0, Skipped: 0
[INFO] All coverage checks have been met.
[INFO] BUILD SUCCESS
[INFO] Total time:  39.013 s
[INFO] Finished at: 2026-11-11T22:37:51Z
```

14. ORGANIZACIÓN DEL TRABAJO

La actividad está planteada como taller grupal, pero se desarrolló de forma individual, por lo que no hay participación de otros integrantes que reportar. Las tareas que en un equipo se repartirían (análisis de casos, automatización, entorno y CI, documentación) las realizó el mismo estudiante.

15. REFLEXIÓN FINAL

Lo que más me dejó este taller fue ver que tener todo en verde no significa que la aplicación esté bien. Con 50 pruebas pasando y 100 % de cobertura, bastó probar a mano un nombre con espacios alrededor para encontrar un defecto: el sistema aceptaba " A " y guardaba "A", algo que la regla de 3 caracteres no debería permitir. Lo mismo pasó con la edad decimal. El error no estaba en una parte sin probar, sino en el orden de dos pasos que sí se ejecutaban.

Escribir primero la prueba que falla y después corregir el código me resultó más claro que corregir directamente: la prueba en rojo demuestra que el defecto existe y, cuando pasa a verde, queda vigilando que no vuelva. También entendí en la práctica la pirámide de pruebas: las 34 unitarias corren en unos dos segundos y las 8 de Selenium tardan casi 25, así que tiene sentido que la mayoría de las reglas se verifiquen abajo y el navegador quede para confirmar lo que ve el usuario.

Las dificultades de la sección 10 me mostraron otra cara de la automatización: buena parte del trabajo no está en escribir aserciones, sino en que el entorno sea confiable (versiones de imágenes que no existen, navegadores que tardan en arrancar, permisos de archivos en el servidor de CI). Docker con versiones fijas resolvió casi todo eso y permitió que la misma suite corriera igual en mi equipo y en GitHub Actions.

Como limitación, la aplicación guarda los datos en memoria, así que la suite no evalúa persistencia ni concurrencia, y las pruebas automatizadas no reemplazan la exploración manual, que fue justamente la que encontró los defectos. Para una siguiente versión agregaría una base de datos real con sus pruebas de integración y algunas pruebas de rendimiento de la API.

REFERENCIAS

Docker Inc. (s. f.). *Docker Compose*. https://docs.docker.com/compose/

Fowler, M. (2012). *TestPyramid*. https://martinfowler.com/bliki/TestPyramid.html

Fowler, M. (2013). *PageObject*. https://martinfowler.com/bliki/PageObject.html

GitHub. (s. f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

International Software Testing Qualifications Board. (2023). *Certified Tester Foundation Level (CTFL) v4.0*. https://istqb.org/certifications/certified-tester-foundation-level-ctfl-v4-0/

JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

Mockito. (s. f.). *Mockito framework site*. https://site.mockito.org/

Mountainminds GmbH & Co. KG. (s. f.). *JaCoCo: Java code coverage library. Documentation*. https://www.jacoco.org/jacoco/trunk/doc/

Myers, G. J., Sandler, C. y Badgett, T. (2011). *The art of software testing* (3.ª ed.). John Wiley & Sons.

Postman. (s. f.). *Run collections using Newman CLI*. https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/

Selenium Project. (s. f.-a). *Page object models*. https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/

Selenium Project. (s. f.-b). *Waiting strategies*. https://www.selenium.dev/documentation/webdriver/waits/

Spring. (2024). *Testing Spring Boot applications* (versión 3.3). https://docs.spring.io/spring-boot/3.3/reference/testing/spring-boot-applications.html